

Học máy: Phân loại và Tối ưu Hồi quy Logistic

Quoc Kien

1. Bản chất của Bài toán Phân loại (Classification)

Bản đồ ký hiệu và luồng tính

Ký hiệu	Nghĩa	Cách dùng khi thay số
$\mathbf{x}^{(i)}$	Vector đặc trưng của mẫu thứ i	Nhân với vector hệ số.
β	Vector trọng số, gồm cả bias nếu đã thêm cột 1	Thứ được học/cập nhật.
$z^{(i)}$	Điểm tuyến tính trước sigmoid	$z^{(i)} = \beta^T \mathbf{x}^{(i)}$.
p_i hoặc $\hat{y}^{(i)}$ $y^{(i)}$	Xác suất dự đoán lớp 1 Nhãn thật, 0 hoặc 1	$p_i = \sigma(z^{(i)})$. So với p_i để tính loss/gradient.
J	Cross-entropy loss	Hàm cần tối thiểu hóa.
∇J	Gradient	Dùng trong cập nhật $\beta \leftarrow \beta - \alpha \nabla J$.

Luồng làm bài: tính z , đưa qua sigmoid để có p , áp ngưỡng để dự đoán lớp, tính loss, rồi nếu cần cập nhật thì dùng sai số $p - y$ nhân với đặc trưng.

Trong bài toán phân loại nhị phân (Binary Classification), tập dữ liệu huấn luyện bao gồm n mẫu có dạng $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, trong đó nhãn mục tiêu bị giới hạn trong hai giá trị rời rạc:

$$y^{(i)} \in \{0, 1\}$$

- $y^{(i)} = 1$: Lớp dương tính (Positive class).
- $y^{(i)} = 0$: Lớp âm tính (Negative class).

Tại sao Tuyến tính (Linear Regression) thất bại với Phân loại?

Nếu cố chấp sử dụng mô hình tuyến tính $f_{\beta}(\mathbf{x}) = \beta^T \mathbf{x}$ cho tác vụ phân loại, đường ranh giới quyết định sẽ bị ảnh hưởng nghiêm trọng bởi các điểm dữ liệu dị biệt (Outliers). Khi xuất hiện một điểm dữ liệu nằm rất xa nhưng mang nhãn cực đoan, việc tối thiểu hóa bình phương khoảng cách (MSE) sẽ ép đường thẳng tuyến tính phải xoay một góc rất lớn để bù trừ sai số. Hệ quả là đường ranh giới bị dịch chuyển một cách vô lý, trực tiếp gây ra phân loại sai cho các điểm dữ liệu bình thường ở gần.

Do đó, ta cần một hàm kích hoạt có khả năng ép toàn bộ không gian số thực về khoảng xác suất $(0, 1)$, đó chính là **Hàm số Sigmoid**.

2. Mô hình Logistic Regression & Hàm Sigmoid

Logistic Regression mô hình hóa **xác suất có điều kiện** để một mẫu dữ liệu được phân loại vào lớp 1:

$$P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}) = f_{\beta}(\mathbf{x}^{(i)}) = \sigma(\beta^T \mathbf{x}^{(i)})$$

Hàm số kích hoạt Sigmoid $\sigma(z)$ được định nghĩa toán học là:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Biểu thức tính xác suất cho từng trường hợp nhãn cụ thể của mẫu dữ liệu: * $P(y^{(i)} = 1 \mid \mathbf{x}^{(i)}; \beta) = f_{\beta}(\mathbf{x}^{(i)})$ * $P(y^{(i)} = 0 \mid \mathbf{x}^{(i)}; \beta) = 1 - f_{\beta}(\mathbf{x}^{(i)})$

Công thức Xác suất Hợp nhất

Để thuận tiện cho việc thiết lập hàm toán học tối ưu, hai biểu thức trên được gộp lại thành một phương trình duy nhất dưới dạng tích lũy thừa nhờ tính chất nhị phân của nhãn $y^{(i)} \in \{0, 1\}$:

$$P(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) = [f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}}$$

3. Chứng minh Hàm Mất mát Cross-Entropy bằng MLE

Với giả định các mẫu dữ liệu trong tập huấn luyện độc lập và phân phối cùng quy luật (i.i.d.), hàm lực lượng liên hợp xác suất (Likelihood Function) được thiết lập bằng tích xác suất của tất cả các mẫu:

$$L(\beta) = \prod_{i=1}^n P(y^{(i)} | \mathbf{x}^{(i)}; \beta) = \prod_{i=1}^n [f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}}$$

Theo nguyên lý ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE), ta cần tìm bộ tham số β sao cho giá trị $L(\beta)$ đạt cực đại. Vì hàm logarit là hàm đồng biến đơn điệu, bài toán tương đương với việc tối đa hóa hàm Log-Likelihood:

$$\max_{\beta} L(\beta) \iff \max_{\beta} \log L(\beta) \iff \min_{\beta} -\log L(\beta)$$

Tiến hành lấy logarit tự nhiên và đổi dấu để chuyển sang bài toán tối thiểu hóa hàm mất mát:

$$J(\beta) = -\log L(\beta) = -\sum_{i=1}^n \log \left([f_{\beta}(\mathbf{x}^{(i)})]^{y^{(i)}} [1 - f_{\beta}(\mathbf{x}^{(i)})]^{1-y^{(i)}} \right)$$

Sử dụng tính chất của logarit để đưa số mũ xuống thành hệ số, ta thu được hàm mất mát **Binary Cross-Entropy Loss**:

$$J(\beta) = -\sum_{i=1}^n [y^{(i)} \log f_{\beta}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - f_{\beta}(\mathbf{x}^{(i)}))]$$

4. Chứng minh Tính Lồi (Convex Optimization) bằng Ma trận Hessian

Để các thuật toán tối ưu hóa dựa trên đạo hàm hoạt động tin cậy và không bị kẹt ở các điểm cực tiểu cục bộ, hàm mất mát $J(\beta)$ phải là một hàm lồi. Điều này đồng nghĩa với việc ma trận đạo hàm bậc hai (Hessian Matrix) phải luôn bán xác định dương ($\nabla^2 J(\beta) \succeq 0$) với mọi giá trị của β .

Đạo hàm Bậc một (Gradient) via Chain Rule

Đặt biến trung gian tuyến tính $z^{(i)} = \beta^T \mathbf{x}^{(i)}$ và giá trị dự đoán $\hat{y}^{(i)} = \sigma(z^{(i)}) = f_\beta(\mathbf{x}^{(i)})$. Áp dụng quy tắc chuỗi (Chain Rule) để tính đạo hàm riêng cho từng mẫu dữ liệu:

$$\frac{\partial J^{(i)}(\beta)}{\partial \beta} = \frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \beta}$$

Tính toán chi tiết từng thành phần vi phân:

1. **Thành phần Loss:** $\frac{\partial J^{(i)}}{\partial \hat{y}^{(i)}} = \frac{-y^{(i)}}{\hat{y}^{(i)}} + \frac{1-y^{(i)}}{1-\hat{y}^{(i)}} = \frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1-\hat{y}^{(i)})}$
2. **Thành phần Sigmoid:** $\frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} = \sigma(z^{(i)})(1 - \sigma(z^{(i)})) = \hat{y}^{(i)}(1 - \hat{y}^{(i)})$
3. **Thành phần Tuyến tính:** $\frac{\partial z^{(i)}}{\partial \beta} = \mathbf{x}^{(i)}$

Nhân kết hợp cả ba thành phần lại với nhau để triệt tiêu đại lượng mẫu số:

$$\frac{\partial J^{(i)}(\beta)}{\partial \beta} = \left[\frac{\hat{y}^{(i)} - y^{(i)}}{\hat{y}^{(i)}(1 - \hat{y}^{(i)})} \right] \cdot [\hat{y}^{(i)}(1 - \hat{y}^{(i)})] \cdot \mathbf{x}^{(i)} = (\hat{y}^{(i)} - y^{(i)})\mathbf{x}^{(i)}$$

Đạo hàm Bậc hai (Ma trận Hessian)

Tiếp tục lấy đạo hàm bậc hai bằng cách biệt hóa toán tử Gradient thu được theo vector tham số β^T :

$$H = \nabla^2 J(\beta) = \frac{\partial^2 J(\beta)}{\partial \beta \partial \beta^T} = \sum_{i=1}^n \frac{\partial}{\partial \beta} [(\hat{y}^{(i)} - y^{(i)})\mathbf{x}^{(i)}]$$

Vì nhãn dữ liệu thực tế $y^{(i)}$ là một hằng số cố định và ta đã biết $\frac{\partial \hat{y}^{(i)}}{\partial \beta} = \hat{y}^{(i)}(1 - \hat{y}^{(i)})\mathbf{x}^{(i)}$, biểu thức ma trận Hessian tổng thể rút gọn thành:

$$H = \sum_{i=1}^n \hat{y}^{(i)}(1 - \hat{y}^{(i)})\mathbf{x}^{(i)}(\mathbf{x}^{(i)})^T$$

Biện luận tính lồi: * Vì đầu ra hàm xác suất Sigmoid luôn nằm trong biên hạn hẹp $\hat{y}^{(i)} \in (0, 1)$, hệ số trọng số luôn dương: $\hat{y}^{(i)}(1 - \hat{y}^{(i)}) > 0$. * Bản chất phép nhân ngoài của một vector với chính nó dưới dạng $\mathbf{xx}^T$ luôn tạo dựng nên một ma trận bán xác định dương.

Do tổng các ma trận bán xác định dương là một ma trận bán xác định dương, ta kết luận $H \succeq 0 \forall \beta$. Hàm mục tiêu của Logistic Regression **hoàn toàn là hàm lồi (Convex Function)**.

5. Các Phương pháp Tối ưu hóa Toán học

Thuật toán Gradient Descent (Tối ưu bậc 1)

Thực hiện cập nhật véc-tơ trọng số lặp bằng cách đi ngược hướng với vector độ dốc Gradient hệ số học α :

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}) = \beta^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) \mathbf{x}^{(i)}$$

Công thức cập nhật chi tiết cho từng thành phần chỉ số tham số β_j trong không gian đa chiều:

$$\beta_j^{(t+1)} = \beta_j^{(t)} + \alpha \sum_{i=1}^n (y^{(i)} - f_{\beta}(\mathbf{x}^{(i)})) x_j^{(i)}$$

Phương pháp Newton - Raphson (Tối ưu bậc 2)

Để tìm điểm cực trị của hàm lỗi bằng cách giải hệ phương trình đạo hàm bằng không $\nabla J(\beta) = 0$, phương pháp Newton sử dụng khai triển Taylor bậc hai nhằm mượn thêm thông tin độ cong từ ma trận Hessian.

Công thức cập nhật đa biến sử dụng ma trận nghịch đảo Hessian tại mỗi bước lặp:

$$\beta^{(t+1)} = \beta^{(t)} - H^{-1} \nabla J(\beta^{(t)})$$

Dạng ma trận dùng trong bài thi Newton.

Với ma trận thiết kế $\mathbf{X} \in \mathbb{R}^{n \times p}$, vector xác suất $\mathbf{p} = \sigma(\mathbf{X}\beta)$, và vector nhãn $\mathbf{y}$:

$$\nabla J(\beta) = \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

Đặt ma trận đường chéo:

$$\mathbf{R} = \text{diag}(p_1(1-p_1), p_2(1-p_2), \dots, p_n(1-p_n))$$

thì Hessian là:

$$\nabla^2 J(\beta) = \mathbf{X}^T \mathbf{R} \mathbf{X}$$

và bước Newton có dạng:

$$\beta_{new} = \beta - (\mathbf{X}^T \mathbf{R} \mathbf{X})^{-1} \mathbf{X}^T (\mathbf{p} - \mathbf{y})$$

Để kiểm tra tính lồi, lấy bất kỳ vector $\mathbf{v}$:

$$\mathbf{v}^T \nabla^2 J \mathbf{v} = \mathbf{v}^T \mathbf{X}^T \mathbf{R} \mathbf{X} \mathbf{v} = (\mathbf{X} \mathbf{v})^T \mathbf{R} (\mathbf{X} \mathbf{v}) \geq 0$$

vì mọi phần tử đường chéo $p_i(1 - p_i)$ đều không âm. Đây là lý do Logistic Regression với cross-entropy có nghiệm tối ưu toàn cục, còn Newton's Method có thể tận dụng độ cong để đi rất nhanh gần nghiệm.

So sánh đặc tính vận hành:

Thuật toán Tối ưu	Cơ chế Đạo hàm	Ưu điểm Vận hành	Nhược điểm Khuyết góc
Gradient Descent	Chỉ sử dụng bậc một (∇J)	Chi phí tính toán trên mỗi bước lặp rất thấp ($\mathcal{O}(p)$). Phù hợp với không gian dữ liệu lớn chứa hàng triệu thuộc tính p .	Tốc độ hội tụ chậm (Linear Convergence). Tốn nhiều thời gian thử nghiệm để tìm ra hệ số học α phù hợp.
Newton's Method	Kết hợp bậc một và ma trận bậc hai ($H^{-1} \nabla J$)	Tốc độ hội tụ cực kỳ nhanh nhờ đặc tính hội tụ bậc hai (Quadratic Convergence). Số vòng lặp yêu cầu rất ít và không cần tùy chỉnh hyperparameter hệ số học α .	Chi phí tính toán ma trận nghịch đảo Hessian rất lớn ($\mathcal{O}(p^3)$) tại mỗi bước lặp. Hoàn toàn bất khả thi nếu số lượng thuộc tính p lớn.

6. Kiểm tra Thực thi: Sigmoid và Ranh giới Quyết định

Các công thức phía trên sẽ dễ nhớ hơn nếu gắn với hai trục giác hình học: sigmoid biến một điểm số thực thành xác suất, và ngưỡng $P(y = 1 \mid \mathbf{x}) = 0.5$ tạo ra ranh giới quyết định tuyến tính.

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

np.random.seed(7)
n = 120
X0 = np.random.normal(loc=[-1.2, -0.8], scale=[0.55, 0.55], size=(n // 2, 2))
X1 = np.random.normal(loc=[1.2, 1.0], scale=[0.65, 0.65], size=(n // 2, 2))
X = np.vstack([X0, X1])
y = np.hstack([np.zeros(n // 2), np.ones(n // 2)])
X_design = np.c_[np.ones(n), X]

beta = np.zeros(3)
lr = 0.4
for _ in range(800):
    probs = sigmoid(X_design @ beta)
    grad = X_design.T @ (y - probs) / n
    beta += lr * grad

pred = (sigmoid(X_design @ beta) >= 0.5).astype(int)
accuracy = (pred == y).mean()
print("Beta học được:", np.round(beta, 3))
print("Độ chính xác huấn luyện:", round(float(accuracy), 3))

z = np.linspace(-8, 8, 300)
xx, yy = np.meshgrid(np.linspace(-3, 3, 160), np.linspace(-3, 3, 160))
grid_design = np.c_[np.ones(xx.size), xx.ravel(), yy.ravel()]
surface = sigmoid(grid_design @ beta).reshape(xx.shape)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.8))
ax[0].plot(z, sigmoid(z), color="#4C78A8")
ax[0].axhline(0.5, color="black", linestyle="--", linewidth=1)
ax[0].axvline(0.0, color="black", linestyle="--", linewidth=1)
ax[0].set_title("Sigmoid biến điểm số thành xác suất")
ax[0].set_xlabel("điểm tuyến tính z")
```

```

ax[0].set_ylabel(" $\sigma(z)$ ")
ax[0].grid(alpha=0.25)

contour = ax[1].contourf(xx, yy, surface, levels=20, cmap="RdBu_r",
    ↪ alpha=0.75)
ax[1].scatter(X0[:, 0], X0[:, 1], label="class 0", edgecolor="black",
    ↪ alpha=0.85)
ax[1].scatter(X1[:, 0], X1[:, 1], label="class 1", edgecolor="black",
    ↪ alpha=0.85)
ax[1].contour(xx, yy, surface, levels=[0.5], colors="black", linewidths=2)
ax[1].set_title("Ranh giới xác suất học được")
ax[1].set_xlabel(" $x_1$ ")
ax[1].set_ylabel(" $x_2$ ")
ax[1].legend()
fig.colorbar(contour, ax=ax[1], label=" $P(y=1 \mid x)$ ")
plt.tight_layout()
plt.show()

```

Beta học được: [-0.101 4.325 3.452]

Độ chính xác huấn luyện: 1.0

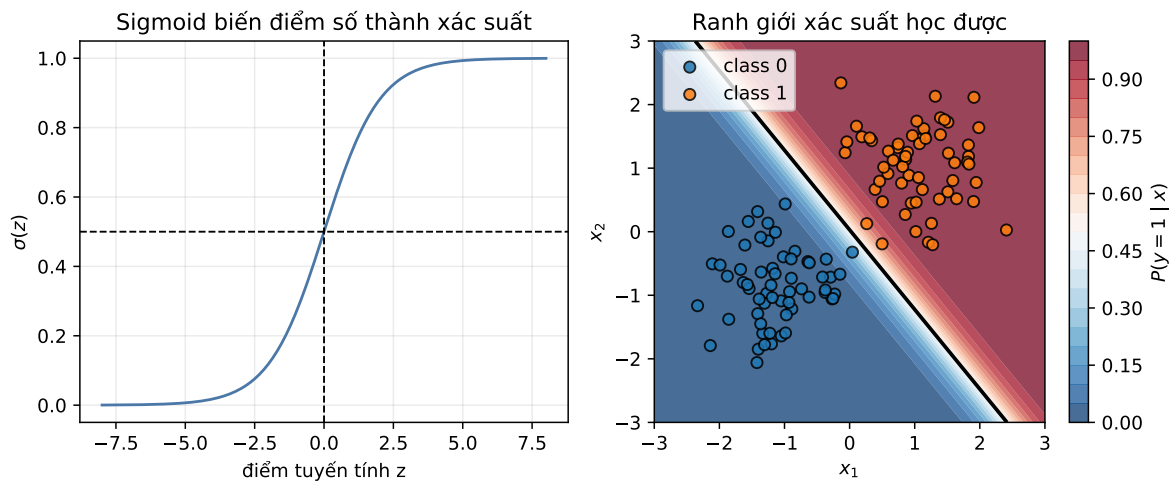


Figure 1: Logistic Regression ánh xạ điểm tuyến tính qua sigmoid và học ranh giới xác suất tuyến tính.

7. Quy trình giải bài Logistic Regression

Một bài Logistic Regression thường chỉ xoay quanh bốn thao tác:

1. Tính điểm tuyến tính

$$z_i = \beta^T \mathbf{x}_i$$

2. Đổi thành xác suất

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}$$

3. Tính loss

$$J_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$$

4. Tính gradient hoặc cập nhật

$$\nabla J = \frac{1}{n} \mathbf{X}^T (\mathbf{p} - \mathbf{y}), \quad \beta_{new} = \beta - \alpha \nabla J$$

Bẫy thường gặp: nếu đang tối thiểu hóa cross-entropy thì dùng $(p - y)$; nếu đang cực đại hóa log-likelihood thì dấu đổi thành $(y - p)$. Hai cách viết đúng nhưng cập nhật phải nhất quán với mục tiêu.

8. Bảng Công thức Thi: Quy trình Thay số cho Logistic Regression

Dùng phần này khi đề bài nhắc đến “phân loại nhị phân”, “xác suất thuộc lớp 1”, “log odds”, hoặc “cross-entropy”.

Nhiệm vụ	Công thức	Cách dùng
Điểm tuyến tính	$z^{(i)} = \beta^T \mathbf{x}^{(i)}$	Nén toàn bộ đặc trưng thành một số thực.
Xác suất	$\hat{y}^{(i)} = \sigma(z^{(i)}) = \frac{1}{1 + e^{-z^{(i)}}}$	Biến điểm số bất kỳ thành xác suất trong $(0, 1)$.
Luật quyết định	$\hat{y}^{(i)} \geq 0.5 \Rightarrow \text{lớp 1}$	Tương đương $z^{(i)} \geq 0$ nếu ngưỡng là 0.5.

Nhiệm vụ	Công thức	Cách dùng
Loss một mẫu	$J^{(i)} = -y^{(i)} \log \hat{y}^{(i)} - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$	Phạt rất nặng khi mô hình tự tin nhưng sai.
Gradient	$\nabla J = \sum_i (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$	Sai số dự đoán nhân với vector đặc trưng.
Hessian	$H = \sum_i \hat{y}^{(i)} (1 - \hat{y}^{(i)}) \mathbf{x}^{(i)} (\mathbf{x}^{(i)})^T$	Bán xác định dương, nên hàm mục tiêu lồi.
Cập nhật Gradient Descent	$\beta \leftarrow \beta - \alpha \nabla J$	Đi ngược chiều gradient của cross-entropy.

Bẫy thường gặp khi thi.

- Logistic Regression là mô hình tuyến tính theo không gian đặc trưng, nhưng xác suất đầu ra phi tuyến vì có sigmoid.
- Không mặc định dùng MSE để suy luận Logistic Regression; cross-entropy xuất phát từ Bernoulli MLE.
- Dấu của $z = \beta^T \mathbf{x}$ quyết định lớp khi ngưỡng là 0.5.
- Độ lớn $|z|$ thể hiện mức tự tin: dương lớn nghĩa là gần lớp 1, âm lớn nghĩa là gần lớp 0.

```
import numpy as np

beta = np.array([-0.4, 1.2, -0.8])
X_demo = np.array([
    [1.0, 0.0, 0.0],
    [1.0, 2.0, 1.0],
    [1.0, -1.0, 2.0],
])
y_demo = np.array([0, 1, 0])
scores = X_demo @ beta
probs = 1 / (1 + np.exp(-scores))
losses = -y_demo * np.log(probs) - (1 - y_demo) * np.log(1 - probs)

print("Điểm z:", np.round(scores, 3))
print("Xác suất:", np.round(probs, 3))
print("Lớp dự đoán:", (probs >= 0.5).astype(int))
print("Cross-entropy từng mẫu:", np.round(losses, 3))
print("Loss trung bình:", round(float(losses.mean()), 3))
```

```
Điểm z: [-0.4  1.2 -3.2]
Xác suất: [0.401 0.769 0.039]
Lớp dự đoán: [0 1 0]
```

Cross-entropy từng mẫu: [0.513 0.263 0.04]
Loss trung bình: 0.272

Dạng bài: chứng minh MLE dẫn tới Binary Cross-Entropy

Khi đề yêu cầu “viết likelihood, log-likelihood, rồi chứng minh tương đương với cross-entropy”, hãy làm theo khung này.

Với $p_i = P(y_i = 1 \mid x_i; \beta) = \sigma(\beta^T \mathbf{x}_i)$:

$$P(y_i \mid x_i; \beta) = p_i^{y_i} (1 - p_i)^{1-y_i}$$

Likelihood toàn bộ dữ liệu:

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$$

Log-likelihood:

$$\ell(\beta) = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Negative log-likelihood:

$$-\ell(\beta) = -\sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Đây chính là binary cross-entropy nếu bỏ hoặc chia thêm hệ số $\frac{1}{n}$. Vì nhân/chia bởi hằng số dương không đổi điểm tối ưu:

$$\arg \max \ell(\beta) = \arg \min [-\ell(\beta)]$$

Gradient của log-likelihood dùng cho gradient ascent:

$$\nabla \ell(\beta) = \sum_{i=1}^n (y_i - p_i) \mathbf{x}_i$$

Gradient của cross-entropy dùng cho gradient descent:

$$\nabla J(\beta) = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) \mathbf{x}_i$$

Hai công thức khác nhau vì một bên tối đa hóa, một bên tối thiểu hóa.

Bộ bài tập mẫu có lời giải

Đề bài. Cho mô hình Logistic Regression có $\beta = (-0.4, 1.2, -0.8)$, trong đó phần tử đầu tiên là bias. Với mẫu $\mathbf{x} = (1, 2, 1)$ và nhãn thật $y = 1$, hãy tính xác suất dự đoán, lớp dự đoán, và loss cross-entropy.

Lời giải từng bước.

1. Tính điểm tuyến tính:

$$z = -0.4 + 1.2(2) - 0.8(1) = 1.2$$

2. Đưa qua sigmoid:

$$\hat{y} = \sigma(1.2) = \frac{1}{1 + e^{-1.2}} \approx 0.768$$

3. Vì $0.768 \geq 0.5$, mô hình dự đoán **lớp 1**.

4. Vì $y = 1$, loss là:

$$J = -\log(0.768) \approx 0.264$$

Kết luận. Mô hình dự đoán đúng lớp 1 với xác suất khoảng 76.8%, và loss thấp vì dự đoán khá tự tin.

Bài 2: Bảng 12 dòng, tính xác suất, loss, accuracy

Đề bài. Cho mô hình $\beta = (-1.0, 0.9, 0.7)$, trong đó phần tử đầu là bias. Với 12 mẫu sau, hãy tính z , xác suất, lớp dự đoán, cross-entropy trung bình, và accuracy.

dòng	x_1	x_2	y
1	0.0	0.0	0
2	0.5	0.2	0
3	1.0	0.4	0
4	1.0	1.0	1
5	1.5	0.8	1
6	2.0	0.5	1
7	-0.5	1.0	0
8	0.2	1.8	1
9	2.5	1.0	1
10	-1.0	0.5	0
11	1.2	-0.2	0
12	0.8	1.5	1

Lời giải. Mỗi dòng làm cùng một quy trình:

$$z = -1.0 + 0.9x_1 + 0.7x_2, \quad \hat{p} = \frac{1}{1 + e^{-z}}$$

Ví dụ dòng 4:

$$z = -1.0 + 0.9(1.0) + 0.7(1.0) = 0.60$$

$$\hat{p} = \sigma(0.60) = \frac{1}{1 + e^{-0.60}} \approx 0.646$$

Vì $\hat{p} \geq 0.5$, dự đoán lớp 1. Vì $y = 1$, loss dòng 4 là:

$$-\log(0.646) \approx 0.437$$

Làm tương tự cho 12 dòng:

dòng	z	$\hat{p}$	lớp dự đoán	loss
1	-1.000	0.269	0	0.313
2	-0.410	0.399	0	0.509
3	0.180	0.545	1	0.787
4	0.600	0.646	1	0.437
5	0.910	0.713	1	0.338
6	1.150	0.760	1	0.275
7	-0.750	0.321	0	0.387
8	0.440	0.608	1	0.497
9	1.950	0.875	1	0.133
10	-1.550	0.175	0	0.192
11	-0.060	0.485	0	0.664
12	0.770	0.684	1	0.380

Tổng loss xấp xỉ:

$$0.313 + 0.509 + \dots + 0.380 = 4.914$$

Cross-entropy trung bình:

$$\bar{J} = \frac{4.914}{12} \approx 0.409$$

Mô hình sai duy nhất ở dòng 3, nên accuracy:

$$\text{Accuracy} = \frac{11}{12} \approx 0.917$$

Kết luận. Bài kiểu này không cần học lại mô hình. Chỉ cần thay số vào z , qua sigmoid, đặt ngưỡng 0.5, rồi tính loss từng dòng.

Bài 3: Một bước cập nhật gradient

Đề bài. Với cùng 12 dòng ở Bài 2, từ $\beta = (-1.0, 0.9, 0.7)$, hãy tính một bước gradient descent với learning rate $\alpha = 0.1$.

Lời giải. Gradient của cross-entropy là:

$$\nabla J = \frac{1}{n} \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y})$$

Từ bảng ở Bài 2, lấy từng sai số xác suất $\hat{p}_i - y_i$, rồi nhân với từng cột của $\mathbf{X}$. Sau khi cộng và chia cho $n = 12$, ta được:

$$\nabla J \approx \begin{pmatrix} 0.0399 \\ -0.0765 \\ -0.1166 \end{pmatrix}$$

Cập nhật:

$$\beta_{new} = \beta - 0.1 \nabla J$$

$$\beta_{new} \approx \begin{pmatrix} -1.0 \\ 0.9 \\ 0.7 \end{pmatrix} - 0.1 \begin{pmatrix} 0.0399 \\ -0.0765 \\ -0.1166 \end{pmatrix} = \begin{pmatrix} -1.0040 \\ 0.9076 \\ 0.7117 \end{pmatrix}$$

Kết luận. Nếu $\hat{p} - y$ dương, mô hình đang dự đoán xác suất lớp 1 quá cao cho dòng đó; gradient sẽ đẩy hệ số theo hướng giảm xác suất đó.